Adam Willis, acc22aw

COM31006 Assignment Report

https://drive.google.com/file/d/1ZShlk-n5qm2pX1fwFQynf0ayUbepV88R/view?usp=sharing
https://youtu.be/fuF2e0eu73w

Introduction

The main objective of this assignment was to create a lightweight image classifier designed for the CPU. To achieve this, a dataset was created and used to explore existing architectures, compare filters, and implement filter selection strategies. Quantitative metrics and qualitative samples were used throughout to evaluate the models used in the project.

Part I: Dataset

Household Items were chosen as the domain to ensure items were accessible and easily photographable; books, bottles, cutlery, mugs, and shoes made up the 5 classes. It was clear that the larger the dataset, the more accurate the results would be. However, due to the manual creation of the dataset, an extremely large dataset was infeasible. Therefore, 400 photographs were taken, 80 per class, with a 80% training, 20% test split. Creation of this dataset was feasible within the project's timeframe, and any images exceeding the required 150 images helped improve the credibility of the results.

Figure I.1: Dataset Samples (de-normalised, no augmentation)

Due to its small size, strategies were implemented to improve the dataset quality. Variables were used during photography to ensure varied images, such as different items, orientations, distances, lighting, and backgrounds, as shown in Figure I.1. Additionally, image augmentation was applied during preprocessing, consisting of randomised horizontal flips and colour jitters.

During preprocessing, after resizing to 128x128, the images were normalised using the ImageNet mean and standard deviation. The existing CNNs used throughout the project were originally trained using these normalisation values. Therefore, aligning the dataset images improved the performance of the pretrained architectures on the custom dataset. The final preprocessing step split the base training set into augmented training data and validation data. Only the training data was augmented, as validation and testing should report results on the original images.

Finally, hashing was used to prove that the dataset was correct and varied. Image byte hashes were calculated for all images, and the training and test sets were compared to ensure no overlap, as duplicate images can affect results. Differences between perceptual hashes were calculated between image pairs to ensure that images in the dataset were not overly similar. A difference of 10 was used as a benchmark for acceptable similarity.

```
Total Comparisons: 25600
Closest Perceptual Similarity: 18
Average Perceptual Similarity: 22.05
-------------------------------------------------
Valid Dataset
```

Figure I.2: Perceptual Hashing Validation Results

Part II: Exploring Existing CNN Architectures

Exploring existing classifiers enabled comparisons when testing the new image classifier, and architectural analysis informed the implementation of filter selection strategies. The models chosen for this project were AlexNet, ResNet50, and MobileNetV2. AlexNet was chosen for its simple architecture and good transfer learning performance. ResNet50 was chosen as a benchmark for classification accuracy. MobileNetV2 was designed to be lightweight, a requirement of the new classifier. It was therefore chosen to be a benchmark for computational efficiency metrics. Additionally, all of these models were pretrained on ImageNet data, meaning the dataset could be preprocessed and used the same way for all three models, simplifying implementation.

Before the classifiers could be evaluated, the final layer was modified to match the number of classes. Then the models were frozen except for the replaced classifier layer. This was done to prevent overfitting, as the pretrained models were too powerful for the dataset. Finally, validation and early stoppage were implemented; early stoppage occurred if validation loss did not drop over a given number of epochs. Early stoppage also prevented overfitting. Validation loss was an effective metric to use as it calculated the model's performance on unseen data, similar to evaluation results.

AlexNet, ResNet50 and MobileNetV2 were evaluated over 5 runs, with an early stoppage tolerance of 1. Figures II.1 and II.2 show selected architectural attributes and the average results from the 5 classification runs.

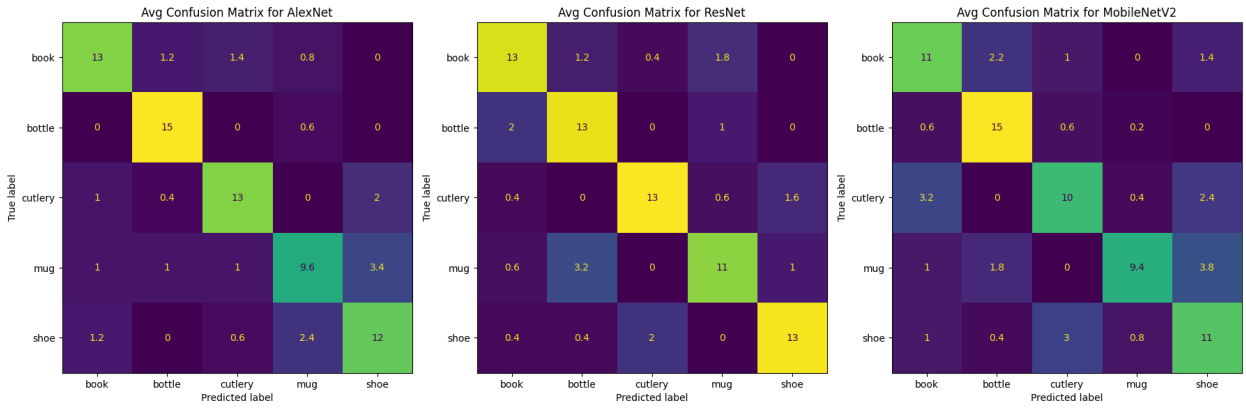

Figure II.1: Confusion Matrices for Existing CNNs

Model	AlexNet	ResNet	MobileNetV2
Accuracy	0.78 ± 0.01	0.79 ± 0.01	0.70 ± 0.01
Macro F1	0.77 ± 0.01	0.79 ± 0.01	0.70 ± 0.02
Total Parameters	57,024,325	23,518,277	2,230,277
Trainable Parameters	20,485	10,245	6,405
Largest Layer	classifier.1.weight	layer4.0.conv2.weight	features.18.0.weight
Largest Trainable Layer	classifier.6.weight	fc.weight	classifier.1.weight

Figure II.2: Results and Attributes of Existing CNNs

As expected, ResNet exhibited the best pretrained filters, achieving the highest average accuracy. However, the dataset's limited size meant that incorrect predictions had a large effect on accuracy. Therefore, AlexNet could be considered as effective as ResNet. MobileNetV2 was chosen for efficiency; its significantly lower parameter count reflects this. Trainable parameters were low due to the freezing of the pretrained model. Interestingly, ResNet50 and MobileNetV2's largest layers were found in Conv sequences, while AlexNet's were found in the classifier head, due to its large fully connected layers. Finally, the confusion matrices and F1 values indicate similar accuracy across classes, providing evidence that the classes were distinct.

Part III: Comparing Classical and Learned Filters

Comparing filters helped visualise how CNNs relate to classic image classification tasks and identify which kind of filter is learned more frequently. Quantitative and qualitative methods were used to compare the pretrained models to classical filters and find the strongest matches.

A classical filter suite was created for comparison, including gradient filters (Sobel), line detectors (Gabor), and smoothing operators (Gaussian). Due to dimension differences, classical filters were resized using bilinear interpolation to match the current learned filters, allowing similarity and distance metrics to be calculated. Cosine similarity, Pearson similarity, L1 distance and L2 distance were calculated for each learned and classical filter pair to build a similarity matrix for each metric. The best matches were then applied to a sample image for qualitative analysis of the filters.

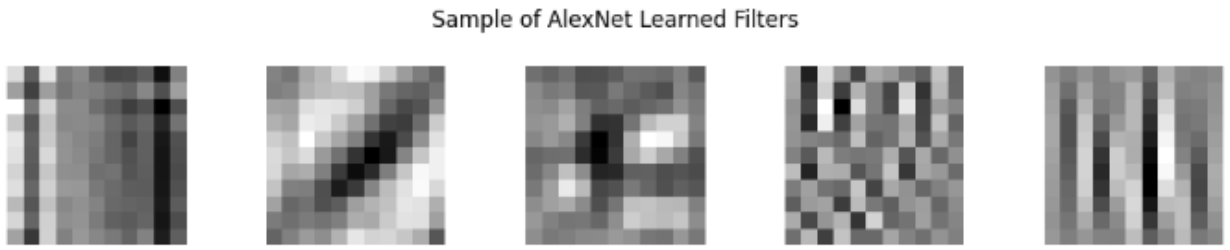

Figure III.1: AlexNet Conv1 Filter Samples

Do some learned filters resemble edge detectors?

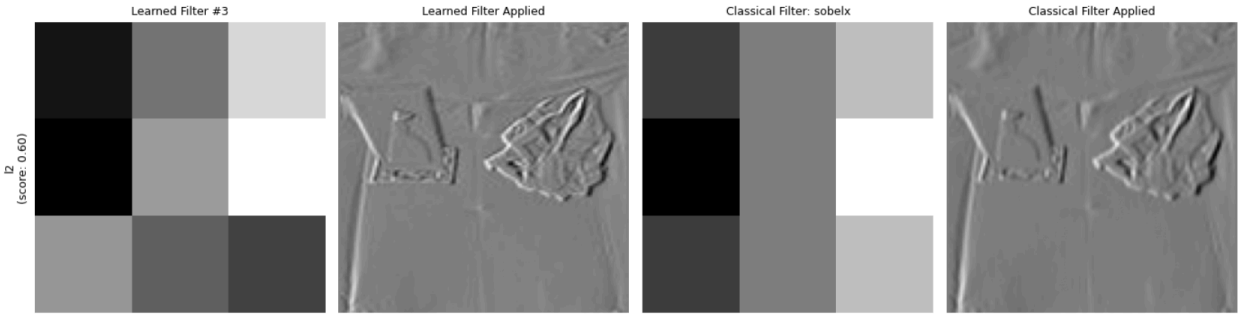

Figure III.2: MobileNetV2 Conv1 Filter #3 and Sobelx applied to a Sample

Figure III.2 shows the closest L2 distance match in MobileNetV2's Conv1 layer; there is a clear visual similarity between learned filter #3 and the Sobelx filter. Applying the filters to a sample image produced similar activations in both products. Therefore, this provided evidence that early CNN layers can learn edge detectors such as Sobelx.

Are there filters that behave similarly to smoothing operators?

```
ResNet Filters Ranked by Pearson Similarity
-------------------------------------------------
Learned Filter              Closest Classical Match      Score
4                           log                          0.9533
19                          gaussian                     0.7872
1                           gabor90                      0.7242
```

Figure III.3: Pearson Similarity Ranking of ResNet50 Conv1 Filters

ResNet's Pearson similarity rankings show that Filter #19's best match was the Gaussian classical filter, a common smoothing operator. This shows that learned filters can exhibit behaviour similar to smoothing filters, and early layers of image classifiers may contain smoothing operators alongside edge detectors and other classical filters.

What differences do you observe?

Although the quantitative metrics and visualisations of the learned and classical filters were very similar for the best matches, some differences were identified. The classical filters were manually implemented and therefore symmetric and centred. The learned filters showed more variance in positions, orientations and gradients. This is likely due to learned filters being optimised through training data rather than being specifically designed to achieve a goal, like classical filters.

Why might deeper-layer filters look less interpretable?

As shown, early classifier layers seem to learn techniques such as edge detection and smoothing through training. Learned filters are directly applied to the original image, so the filtered product remained interpretable and familiar as a classical technique. However, as the layers get deeper, learned filters are applied to the filter products, rather than the original image. Therefore, learned filters in deeper layers learn to activate on more abstract features and become less visually interpretable.

Part IV: Novel Architecture via Filter Selection

Filter selection strategies were used to create a new image classifier from an existing model. AlexNet was chosen for its simple architecture and strong transfer learning performance.

To produce the novel classifier FilterNet, multiple filter selection strategies with different Conv2 filter retain percentages (30% reduction of filters or higher) were applied to AlexNet to identify which filter selection strategy and retain percentage pair produced the best FilterNet version. L1 Norm, L2 Norm, and Random were chosen as the filter selection strategies to test.

In Part III, learned filters were shown to resemble classical filters, including techniques such as edge detection and smoothing. A learned filter that has a good match with a classical filter likely contributes to image classification, and therefore, the model optimises that filter's parameters. The norm-based strategies leverage this by ranking the filters by weight and removing low-ranked filters according to the retain percentage. Random was included to identify whether norm-based methods were more effective than random filter removal.

What modifications were required in the subsequent layers?

After filter selection, the output channels of Conv2 and the input channels of Conv3 did not match; it was necessary to modify those layers to reconcile the layer dimensions and ensure that the correct weights were cloned before evaluation. This allowed FilterNet to remain executable and reduced model overhead; zeroing filter weights after selection would change accuracy without improving computational efficiency.

Additionally, AlexNet's classifier head was modified to reduce the number of fully-connected layers. One of the main aims of the new classifier was to focus on efficiency and CPU

performance, and removing a fully-connected layer supported this by decreasing AlexNet's total parameters, memory space, and computation costs.

Layer	Shape / Params / Status (AlexNet)	Shape / Params / Status (FilteredNet_frozen)	Shape / Params / Status (FilteredNet_trainable)
classifier.1.bias	(4096,) / 4,096 / Frozen	(1024,) / 1,024 / Frozen	(1024,) / 1,024 / Trainable
classifier.1.weight	(4096, 9216) / 37,748,736 / Frozen	(1024, 9216) / 9,437,184 / Frozen	(1024, 9216) / 9,437,184 / Trainable
classifier.4.bias	(4096,) / 4,096 / Frozen	(5,) / 5 / Trainable	(5,) / 5 / Trainable
classifier.4.weight	(4096, 4096) / 16,777,216 / Frozen	(5, 1024) / 5,120 / Trainable	(5, 1024) / 5,120 / Trainable
classifier.6.bias	(5,) / 5 / Trainable	-	-
classifier.6.weight	(5, 4096) / 20,480 / Trainable	-	-

Figure IV.1: Classifier Layer Attributes for AlexNet and FilterNet

Can the modified architecture be used without retraining?

FilterNet evaluation was completed using a frozen (apart from classifier) and trainable (apart from Conv1 and Conv2) FilterNet version to identify whether the remaining pretrained filters could maintain model accuracy, or if fine-tuning was necessary to offset the accuracy decrease caused by filter selection.

Filter Selection Strategy	Accuracy / Macro F1 (FilteredNet_frozen)	Accuracy / Macro F1 (FilteredNet_trainable)
L1 (retained 70%)	0.50 ± 0.06 / 0.49 ± 0.06	0.67 ± 0.01 / 0.67 ± 0.01
L1 (retained 60%)	0.43 ± 0.03 / 0.42 ± 0.03	0.62 ± 0.02 / 0.61 ± 0.02
L1 (retained 50%)	0.38 ± 0.09 / 0.36 ± 0.10	0.55 ± 0.08 / 0.54 ± 0.08
L1 (retained 40%)	0.29 ± 0.09 / 0.27 ± 0.09	0.57 ± 0.02 / 0.56 ± 0.03
L1 (retained 30%)	0.28 ± 0.02 / 0.27 ± 0.03	0.45 ± 0.04 / 0.43 ± 0.05
L2 (retained 70%)	0.61 ± 0.06 / 0.60 ± 0.07	0.74 ± 0.03 / 0.73 ± 0.03
L2 (retained 60%)	0.60 ± 0.03 / 0.59 ± 0.04	0.74 ± 0.04 / 0.73 ± 0.05
L2 (retained 50%)	0.63 ± 0.03 / 0.62 ± 0.03	0.69 ± 0.02 / 0.68 ± 0.03
L2 (retained 40%)	0.58 ± 0.05 / 0.56 ± 0.06	0.63 ± 0.05 / 0.62 ± 0.05
L2 (retained 30%)	0.52 ± 0.09 / 0.48 ± 0.10	0.64 ± 0.06 / 0.63 ± 0.07
RANDOM (retained 70%)	0.61 ± 0.00 / 0.60 ± 0.00	0.65 ± 0.00 / 0.62 ± 0.00
RANDOM (retained 60%)	0.62 ± 0.00 / 0.62 ± 0.00	0.70 ± 0.00 / 0.69 ± 0.00
RANDOM (retained 50%)	0.62 ± 0.00 / 0.60 ± 0.00	0.70 ± 0.00 / 0.68 ± 0.00
RANDOM (retained 40%)	0.57 ± 0.00 / 0.57 ± 0.00	0.62 ± 0.00 / 0.62 ± 0.00
RANDOM (retained 30%)	0.23 ± 0.00 / 0.18 ± 0.00	0.61 ± 0.00 / 0.59 ± 0.00

Figure IV.2: Accuracy Results for FilterNet Versions

Figure IV.2 shows a clear improvement in accuracy for the trainable FilterNet for all implemented strategies. The new model was functional and could be used without retraining earlier layers, but retraining allowed FilterNet to regain the performance loss caused by filter selection.

Part V: Comparison and Evaluation

The best-performing FilterNet variant was produced using the L2 Norm filter selection strategy with 70% filter retention; 30% of Conv2 filters were removed. For comparisons and evaluations, that version of FilterNet will be used.

Did predictions change significantly?

Class predictions maintained similar high-level patterns, as shown in Figures V.1 and V.2, suggesting that the filter selection process did not change general classification behaviour.

While accuracy was affected, general model predictions were likely unchanged between AlexNet and FilterNet due to shared pretrained filters in earlier layers.

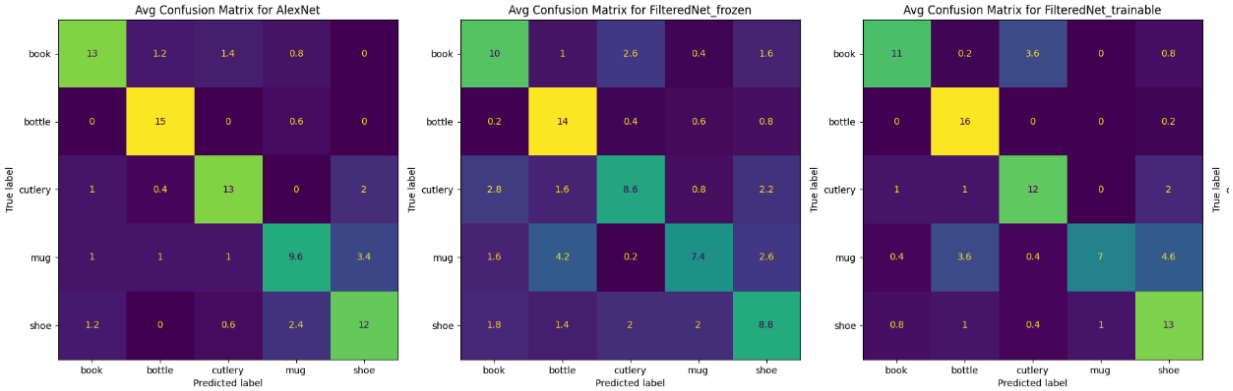

Figure V.1: Confusion Matrices for AlexNet and FilterNet

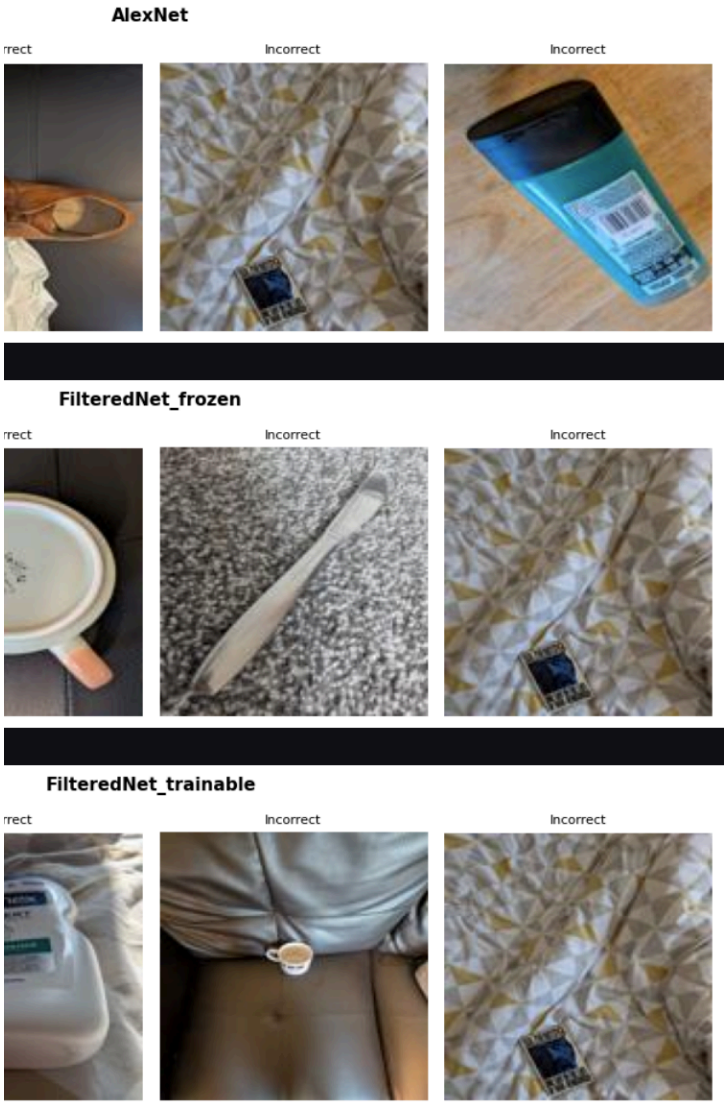

Figure V.2: Shared Incorrect Predictions between AlexNet and FilterNet

Is the network robust to filter removal?

Model	AlexNet	FilteredNet_trainable	ResNet
Total Parameters	57,024,325	11,619,723	23,518,277
Trainable Parameters	20,485	11,381,893	10,245
Largest Layer	classifier.1.weight	classifier.1.weight	layer4.0.conv2.weight
Largest Trainable Layer	classifier.6.weight	classifier.1.weight	fc.weight
Accuracy	0.78 ± 0.01	0.74 ± 0.03	0.79 ± 0.01
Macro F1	0.77 ± 0.01	0.73 ± 0.03	0.79 ± 0.01

Figure V.3: Accuracy and Macro F1 Comparisons

Figure V.3 shows that the trainable FilterNet maintained competitive predictions, with a score similar to the original AlexNet and only slightly lower than ResNet50's accuracy, the benchmark for performance for this project. This indicates that AlexNet can be robust to filter removal. For this project, the large parameter count of the original architecture may have been unnecessary for the fairly limited dataset, leading to filters that did not affect the model when removed.

What trade-offs did you observe?

Trade-offs of the new classifier included the discussed drop in accuracy with an increased standard deviation, indicating a more volatile classifier. However, one of the main requirements was improving efficiency and CPU performance, and FilterNet achieved this goal.

Model	AlexNet	FilteredNet_trainable	MobileNetV2
Total Parameters	57,024,325	11,619,723	2,230,277
Trainable Parameters	20,485	11,381,893	6,405
Largest Layer	classifier.1.weight	classifier.1.weight	features.18.0.weight
Largest Trainable Layer	classifier.6.weight	classifier.1.weight	classifier.1.weight
Accuracy	0.78 ± 0.01	0.74 ± 0.03	0.70 ± 0.01
Macro F1	0.77 ± 0.01	0.73 ± 0.03	0.70 ± 0.02
FLOPs	501,519,744	349,907,584	195,600,896
Average Inference Time	11.14	4.35	10.78

Figure V.4: FLOPs and CPU Inference Time for AlexNet, FilterNet and MobileNetV2

Figure V.4 shows that FilterNet reduced AlexNet's FLOPs by ~30% and CPU inference time by ~60%. This demonstrates that FilterNet is more computationally efficient than AlexNet and therefore more lightweight and CPU-friendly. Interestingly, FilterNet also had a quicker average CPU inference time than MobileNetV2, the benchmark for lightweight performance in this project. Overall, FilterNet provides a lightweight, CPU-efficient architecture in exchange for decreased accuracy and increased classification volatility.

Conclusion

The project's main objective was achieved. AlexNet underwent filter selection to produce a new, lightweight, and efficient image classifier, indicated by the FLOPs and inference time reduction.

FilterNet also showed competitive accuracy (down ~5%) when compared to larger models such as ResNet.

Although the dataset underwent various strategies to improve its quality, its limitations cannot be ignored. Future work for this project could include repeating experiments with a larger dataset and identifying whether results align with the project's findings. From there, more filter selection strategies could be tested.